

CLASSIFICATION vs REGRESSION

Both classification and regression are types of supervised learning — you train on labelled examples. The key difference is in what you predict:

Regression → Predict Numbers

Examples:

House price = ₹45 lakhs

Temperature = 32°C

Salary = ₹8 LPA

Output is a NUMBER.

Classification → Predict Categories

Examples:

Spam or Not Spam

Pass or Fail

Disease or Healthy

Fraud or Not Fraud

Output is a CATEGORY.

Problem	Type
Predict salary	Regression
Predict house price	Regression
Predict if student passes	Classification
Predict if email is spam	Classification
Predict diabetes yes/no	Classification

```
# ■ Classification vs Regression Output Comparison ■
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
# Same toy data — study hours vs result
X = np.array([[1],[2],[3],[4],[5],[6],[7],[8]]) # hours studied
y_reg = np.array([30, 40, 45, 55, 65, 70, 80, 90]) # exam score (regression)
y_cls = np.array([ 0, 0, 0, 0, 1, 1, 1, 1]) # pass/fail (classification)
# --- Regression ---
lin = LinearRegression()
lin.fit(X, y_reg)
print('Regression output:', lin.predict([[5.5]])) # → [67.14] (a number)
# --- Classification ---
log = LogisticRegression()
log.fit(X, y_cls)
print('Classification output:', log.predict([[5.5]])) # → [1] (a label)
print('Probability output :', log.predict_proba([[5.5]])) # → [[0.21 0.79]]
# KEY DIFFERENCE:
# Regression → 67.14 (a score)
# Classification → 1 (pass) with 79% confidence
```

```
Regression output: [67.76785714]
Classification output: [1]
Probability output : [[0.2368932 0.7631068]]
```

Can a straight line predict YES or NO properly?

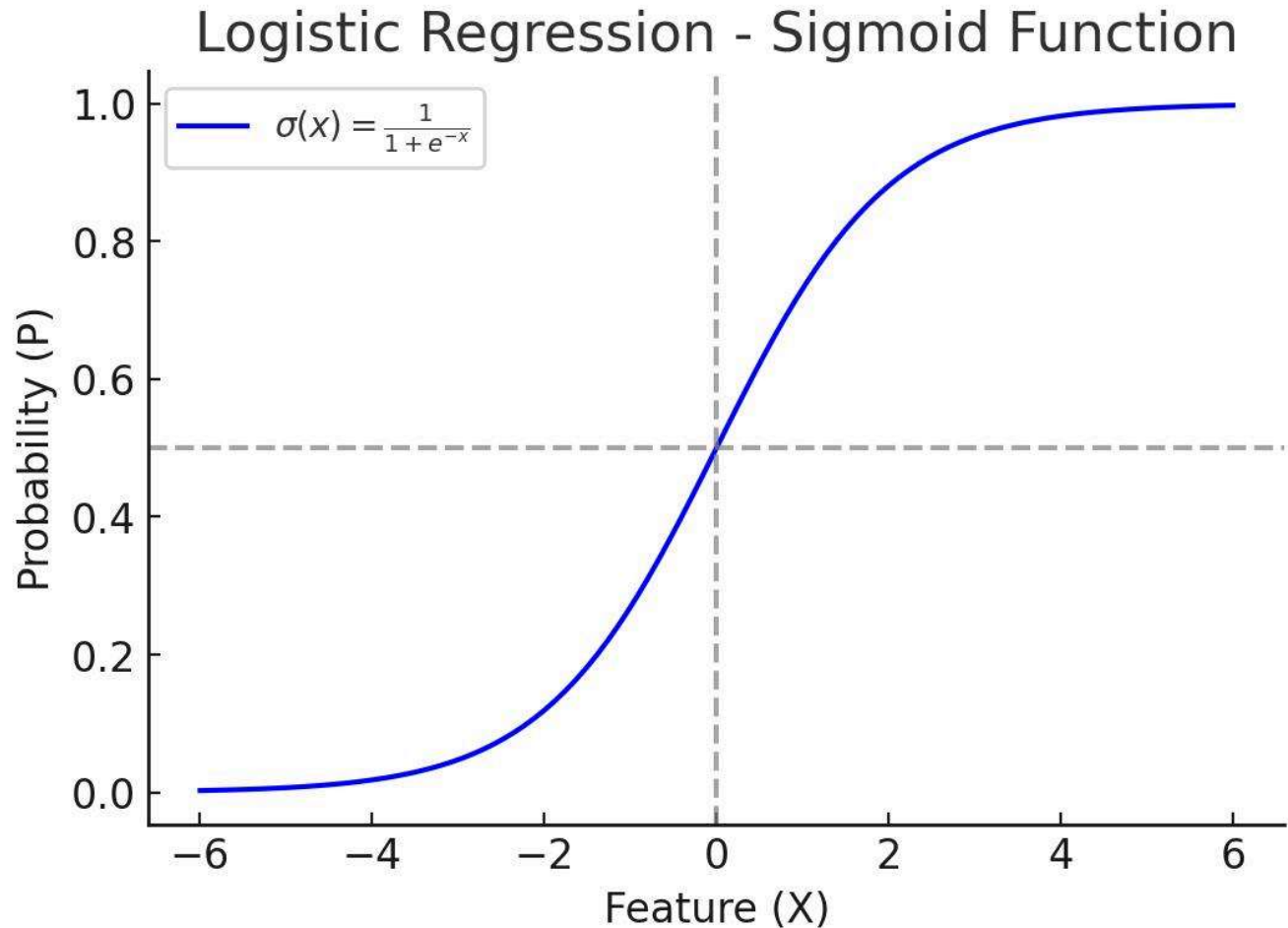
As per our definition and data we have till now it is saying that regression is going to give outputs as numbers where as clasification gives as yes/no

in that case we need one person who actually can convert numners into probabilities instead of line

Despite the name, Logistic Regression is a classification algorithm, not a regression one. It predicts the probability that something belongs to a class (e.g. 'spam' or 'not-spam')

THE SIGMOID CURVE

The sigmoid function takes any real number as input and squashes it into the range [0, 1]. This output is interpreted as a probability.



X in the image represents output prediction.

if x is large positive number $\rightarrow \sigma(x) \rightarrow 1.0$ (very likely YES) • If x is a large negative number $\rightarrow \sigma(x) \rightarrow 0.0$ (very likely NO) • If $z = 0 \rightarrow \sigma(x) = 0.5$ (uncertain — right on the fence)

Output Probability	Meaning
0.1	Very unlikely
0.4	Maybe No
0.7	Maybe Yes
0.95	Almost Yes

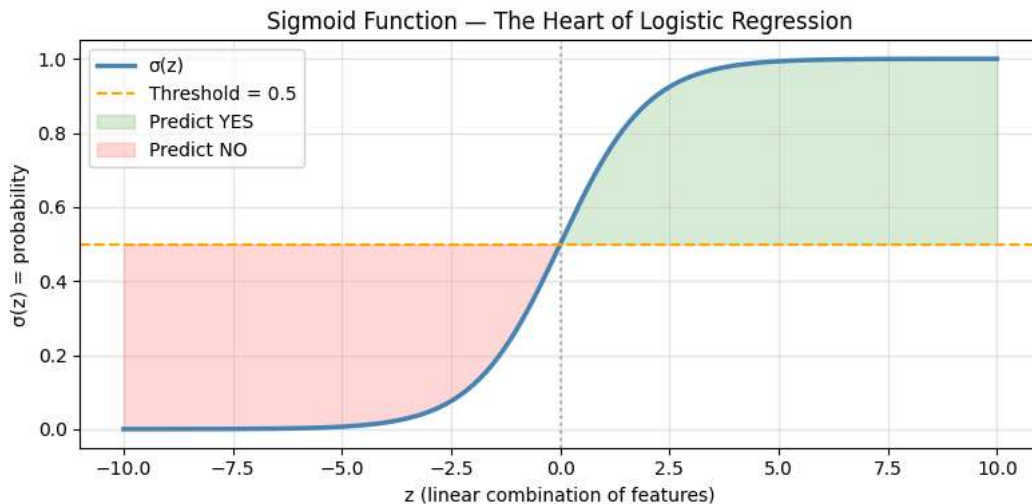
According to above diagram and explanation what sigma curve is going to if number is big we are going to predict it as yes or no.

what if number is in middle or uncertain order for example like in above table , if we get 0.73 how we are going to decide whether it is yes or no.

to understand that we need to know few rules

```
# ■ Plot and Explore the Sigmoid ■
import numpy as np
import matplotlib.pyplot as plt
# ■ Define the sigmoid function ■
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
z = np.linspace(-10, 10, 300) # 300 evenly spaced values from -10 to 10
prob = sigmoid(z) # Apply sigmoid to every value
plt.figure(figsize=(8, 4))
```

```
plt.plot(z, prob, color='steelblue', linewidth=2.5, label='σ(z)')
plt.axhline(y=0.5, color='orange', linestyle='--', label='Threshold = 0.5')
plt.axvline(x=0, color='gray', linestyle=':', alpha=0.6)
plt.fill_between(z, prob, 0.5, where=(prob > 0.5), alpha=0.15, color='green', label='Predict YES')
plt.fill_between(z, prob, 0.5, where=(prob < 0.5), alpha=0.15, color='red', label='Predict NO')
plt.xlabel('z (linear combination of features)')
plt.ylabel('σ(z) = probability')
plt.title('Sigmoid Function — The Heart of Logistic Regression')
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
# Try it: sigmoid(0) = 0.5, sigmoid(5) = 0.993, sigmoid(-5) = 0.007
print(f'sigmoid(0) = {sigmoid(0):.4f}') # 0.5000
print(f'sigmoid(5) = {sigmoid(5):.4f}') # 0.9933
print(f'sigmoid(-5) = {sigmoid(-5):.4f}') # 0.0067
```



```
sigmoid(0) = 0.5000
sigmoid(5) = 0.9933
sigmoid(-5) = 0.0067
```

Decision Threshold

The sigmoid gives us a probability (e.g., 0.73). We need a rule to convert that into a class label. The default rule is called the decision threshold:

Probability > 0.5 → YES

Probability < 0.5 → NO

Probability	Prediction
0.82	Spam
0.23	Not Spam
0.91	Disease
0.15	Healthy

NOTE: Threshold is Tunable

0.5 is the default but not always best. For medical diagnosis (detecting cancer), you might lower the threshold to 0.3 — better to flag a false alarm than to miss a real case. This is the precision–recall trade-off. we will get better understanding on recall trade off and precision on further examples and topics.

Real-World Example: Will This Email Be Spam?

Logistic Regression uses features (columns) of your data to compute z , then applies $\sigma(z)$. Example features for spam detection:

- Number of exclamation marks !!!
- Does the subject contain the word 'FREE' or 'WIN'?
- Sender domain (.ru, .xyz) — known spam source?
- Email length — very short emails are often spam
- Number of links — spammy emails pack in lots of URLs

The model learns the weights (how important each feature is) from thousands of labelled emails during training. After training, it can score new emails in milliseconds.

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split

# Toy binary classification dataset
X, y = make_classification(n_samples=500, n_features=5,
                          weights=[0.7, 0.3], # 70% class-0, 30% class-1 (slight imbalance)
                          random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)
probs = model.predict_proba(X_test)[:, 1] # Raw probabilities for class 1
# ■ Try different thresholds ■
print(f'{'Threshold':>10} {'Precision':>10} {'Recall':>8} {'F1':>6}')
print('-' * 45)
for threshold in [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    preds = (probs >= threshold).astype(int) # Apply our threshold
    p = precision_score(y_test, preds, zero_division=0) # from all predicted to real
    r = recall_score(y_test, preds) # from all positive to got
    f = f1_score(y_test, preds, zero_division=0) # balance btw p n r
    print(f'{'threshold':>10.1f} {'p':>10.3f} {'r':>8.3f} {'f':>6.3f}')
```

OUTPUT (approximate):

Threshold	Precision	Recall	F1
0.2	0.556	0.987	0.713 ← high recall, low precision
0.5	0.762	0.763	0.762 ← balanced default
0.8	0.941	0.421	0.583 ← high precision, low recall

Threshold	Precision	Recall	F1
0.2	0.696	0.941	0.800
0.3	0.762	0.941	0.842
0.4	0.882	0.882	0.882
0.5	0.882	0.882	0.882
0.6	0.903	0.824	0.862
0.7	0.903	0.824	0.862
0.8	0.958	0.676	0.793

Measuring Accuracy for Classification

When you train a classifier you need to measure how good it is. This is more nuanced than regression because you need to distinguish between types of mistakes.

To understand this topic well and better let's start with a medical example, as we can make us to ask many questions. Think in shoe of both patient and Doctor

Did the patient actually have diabetes?

Confusion Matrix

before going to story based a small def or explanation about confusion matrix is needed

A confusion matrix is a 2x2 table (for binary classification) that shows the four possible outcomes when comparing predicted vs actual labels. Think of it as a scoreboard for your model.

Confusion Matrix

	Predicted: YES	Predicted: NO
Actual: YES	TP True Positive (Correct YES)	FP False Positive (Wrong – said YES)
Actual: NO	FN False Negative (Missed – said NO)	TN True Negative (Correct NO)

Actual	Predicted	Result
Sick	Sick	True Positive
Healthy	Healthy	True Negative
Healthy	Sick	False Positive
Sick	Healthy	False Negative

Visual Confusion Matrix

	Predicted NO	Predicted YES
Actual NO	TN	FP
Actual YES	FN	TP

Lets check and understand above concept better with better story n explanation

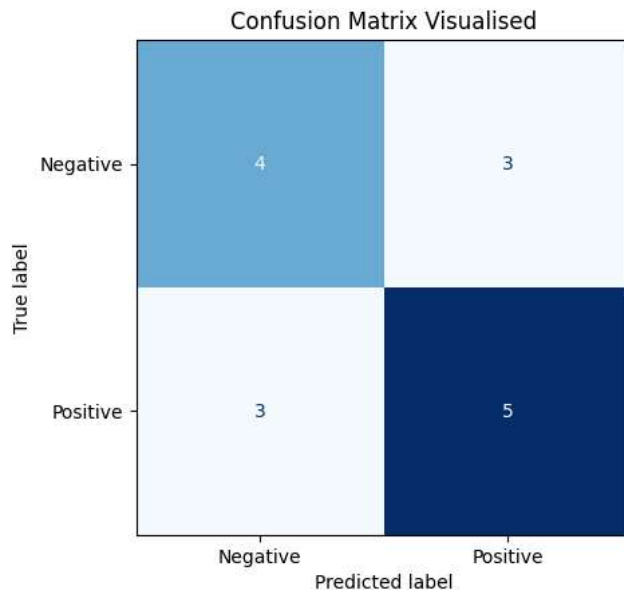
You are a doctor reviewing cancer test results. You tested 100 patients; 20 actually have cancer, 80 are healthy. Your model predicted:

- 18 of the 20 cancer patients as Positive (TP = 18)
- 2 cancer patients were missed — predicted Negative (FN = 2)
- 5 healthy people were wrongly predicted Positive (FP = 5)
- 75 healthy people correctly predicted Negative (TN = 75)

```
# ■ Confusion Matrix: Manual + sklearn ■
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
# Suppose we have these ground truth and predicted labels
y_true = [1,1,1,1,1, 0,0,0,0,0, 1,0,1,0,1] # actual labels
y_pred = [1,1,1,0,0, 0,0,0,1,1, 1,0,0,1,1] # model predictions
# ■ Method 1: Manual calculation ■
y_true = np.array(y_true)
y_pred = np.array(y_pred)
TP = ((y_pred == 1) & (y_true == 1)).sum()
TN = ((y_pred == 0) & (y_true == 0)).sum()
FP = ((y_pred == 1) & (y_true == 0)).sum()
FN = ((y_pred == 0) & (y_true == 1)).sum()
print(f'TP={TP}, TN={TN}, FP={FP}, FN={FN}')
print(f'Accuracy = (TP+TN)/Total = ({TP}+{TN})/{len(y_true)} = {(TP+TN)/len(y_true):.2%}')
print(f'Precision = TP/(TP+FP) = {TP}/{TP+FP} = {TP/(TP+FP):.2%}')
print(f'Recall = TP/(TP+FN) = {TP}/{TP+FN} = {TP/(TP+FN):.2%}')
# ■ Method 2: sklearn does it all ■
cm = confusion_matrix(y_true, y_pred)
print('Confusion matrix:\n', cm)
# ■ Visual plot ■
disp = ConfusionMatrixDisplay(cm, display_labels=['Negative', 'Positive'])
```

```
disp.plot(cmap='Blues', colorbar=False)
plt.title('Confusion Matrix Visualised'); plt.show()
```

```
TP=5, TN=4, FP=3, FN=3
Accuracy = (TP+TN)/Total = (5+4)/15 = 60.00%
Precision = TP/(TP+FP) = 5/8 = 62.50%
Recall = TP/(TP+FN) = 5/8 = 62.50%
Confusion matrix:
[[4 3]
 [3 5]]
```



Precision, Recall, F1-Score & Imbalanced Data

100 patients — 20 have cancer, 80 are healthy. Your model predicted 18 cancer patients correctly (TP=18), missed 2 (FN=2), and falsely flagged 5 healthy patients (FP=5), while correctly clearing 75 (TN=75).

Metric	Formula	Example Calculation	Result	Simple Meaning
Accuracy	$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$	$(18 + 75) / 100$	93%	Out of all predictions, how many were correct?
Precision	$\text{Precision} = \frac{TP}{TP+FP}$	$18 / (18 + 5)$	78%	When the model said "YES", how often was it correct?
Recall	$\text{Recall} = \frac{TP}{TP+FN}$	$18 / (18 + 2)$	90%	Out of all actual YES cases, how many did the model catch?
F1-Score	$\text{F1-Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$	$2 * (0.78 * 0.90) / (0.78 + 0.90)$	83.6%	Balance between Precision and Recall

Why Accuracy Alone Can Mislead

Imbalanced Data Example: Suppose only 1% of transactions are fraudulent. A model that predicts 'NOT FRAUD' for every single transaction achieves 99% accuracy — yet it catches zero frauds! This is why we need Precision, Recall, and F1.

Accuracy is Misleading on Imbalanced Datasets

Rule of thumb: If one class makes up less than 20% of the data, use F1-score or AUC instead of plain accuracy. Always look at the confusion matrix, not just a single number

Which Metric to Prioritise?

Use Case	Best Metric	Why this metric fits
Cancer Screening	Recall	Missing a real cancer case is very dangerous, so we want to catch as many real positive cases as possible.
Spam Filter	Precision	Marking a normal email as spam is annoying, so we want the emails labeled "spam" to be truly spam.
General Purpose	F1-Score	It gives a balanced score by considering both Precision and Recall together.
Ranking / Comparison	AUC-ROC	It helps compare models without fixing one threshold, so it is useful for overall separation performance.

Recall → "Do not miss real positive cases"

Precision → "Do not wrongly mark negatives as positive"

F1-score → "Balance both"

AUC-ROC → "Compare models overall"

```
# ■ Precision, Recall, F1 - Balanced vs Imbalanced ■
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score, accuracy_score
from sklearn.model_selection import train_test_split
import numpy as np

# ■ Balanced dataset (50/50) ■
X_bal, y_bal = make_classification(n_samples=1000, weights=[0.5, 0.5], random_state=1)
# ■ Imbalanced dataset (99/1) ■
X_imb, y_imb = make_classification(n_samples=1000, weights=[0.99, 0.01], random_state=1)

def evaluate(X, y, label):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    model = LogisticRegression(max_iter=1000).fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    print(f'\n=== {label} ===')
    print(f'Accuracy : {acc:.2%} ← {'MISLEADING!' if f1 < 0.5 else 'OK'}')
    print(f'F1 Score : {f1:.2%} ← Real performance')
    print(classification_report(y_test, y_pred, zero_division=0))
evaluate(X_bal, y_bal, 'balanced data')
evaluate(X_imb, y_imb, 'imbalanced data')
# ■ Fix imbalanced: use class_weight='balanced' ■
X_train, X_test, y_train, y_test = train_test_split(X_imb, y_imb, test_size=0.2, random_state=42)

model_fixed = LogisticRegression(class_weight='balanced', max_iter=1000)

model_fixed.fit(X_train, y_train)

y_pred_fixed = model_fixed.predict(X_test)

print('\n=== After class_weight=balanced ===')
print(classification_report(y_test, y_pred_fixed, zero_division=0))
```

```
=== balanced data ===
Accuracy : 80.50% ← OK
F1 Score : 80.00% ← Real performance
```

	precision	recall	f1-score	support
0	0.84	0.78	0.81	106
1	0.77	0.83	0.80	94
accuracy			0.81	200
macro avg	0.81	0.81	0.80	200
weighted avg	0.81	0.81	0.81	200

```
=== imbalanced data ===
Accuracy : 97.50% ← MISLEADING!
F1 Score : 0.00% ← Real performance
```

	precision	recall	f1-score	support
0	0.98	0.99	0.99	196
1	0.00	0.00	0.00	4
accuracy			0.97	200
macro avg	0.49	0.50	0.49	200
weighted avg	0.96	0.97	0.97	200

```
=== After class_weight=balanced ===
```

	precision	recall	f1-score	support
0	0.98	0.89	0.93	196
1	0.04	0.25	0.07	4
accuracy			0.88	200
macro avg	0.51	0.57	0.50	200
weighted avg	0.96	0.88	0.92	200

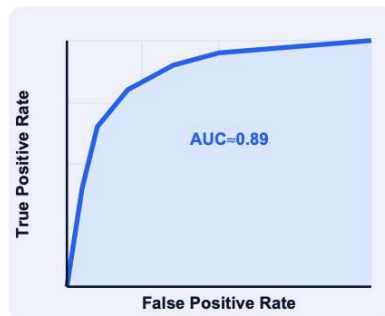
ROC CURVE & AUC

The ROC Curve plots True Positive Rate (Recall) vs False Positive Rate across every possible threshold — giving a complete picture of your model's discriminating power. AUC (Area Under Curve) is a single number summarising the whole curve.

How well model separates YES and NO classes?

Axis	Meaning
X-axis	False alarms
Y-axis	Correct positive detections

AUC Score	Meaning
0.5	Random guessing
0.7	Good
0.8	Very good
0.9+	Excellent



AUC Interpretation

1.00 → Perfect model
 0.90 → Excellent
 0.80 → Good
 0.70 → Fair
 0.60 → Poor
 0.50 → Random guess

The **diagonal dashed line** represents a model that guesses randomly — AUC = 0.5. Your model's curve should always be above it.

Higher AUC means model separates classes better.

```
# ■ ROC Curve: Compare Two Models ■
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
X, y = make_classification(n_samples=800, n_features=10, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

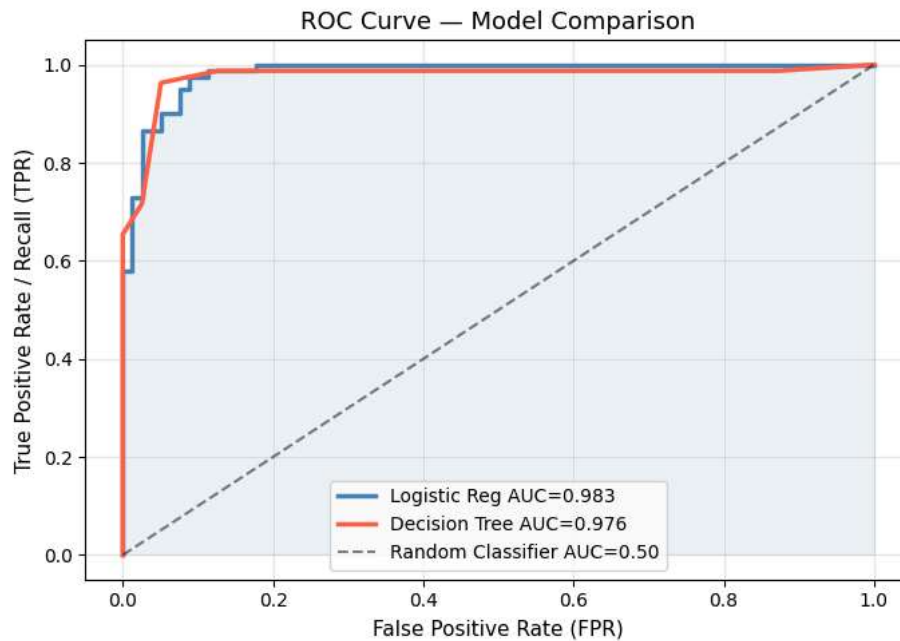
```
# Model 1: Logistic Regression
lr = LogisticRegression().fit(X_train, y_train)
prob_lr = lr.predict_proba(X_test)[:, 1]
```

```
# Model 2: Decision Tree (for comparison)
dt = DecisionTreeClassifier(max_depth=4).fit(X_train, y_train)
prob_dt = dt.predict_proba(X_test)[:, 1]
```

```
# Compute ROC curves
fpr_lr, tpr_lr, _ = roc_curve(y_test, prob_lr)
fpr_dt, tpr_dt, _ = roc_curve(y_test, prob_dt)
auc_lr = roc_auc_score(y_test, prob_lr)
auc_dt = roc_auc_score(y_test, prob_dt)
```

```
# Plot both on same chart
plt.figure(figsize=(7, 5))
plt.plot(fpr_lr, tpr_lr, lw=2.5, color='steelblue', label=f'Logistic Reg AUC={auc_lr:.3f}')
plt.plot(fpr_dt, tpr_dt, lw=2.5, color='tomato', label=f'Decision Tree AUC={auc_dt:.3f}')
plt.plot([0,1],[0,1], 'k--', alpha=0.5, label='Random Classifier AUC=0.50')
```

```
plt.fill_between(fpr_lr, tpr_lr, alpha=0.1, color='steelblue')
plt.xlabel('False Positive Rate (FPR)', fontsize=11)
plt.ylabel('True Positive Rate / Recall (TPR)', fontsize=11)
plt.title('ROC Curve — Model Comparison', fontsize=13)
plt.legend(fontsize=10); plt.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



```
print(f'Logistic Regression AUC: {auc_lr:.4f}')
print(f'Decision Tree AUC: {auc_dt:.4f}')
print('Higher AUC = better model at separating classes')
```

```
Logistic Regression AUC: 0.9833
Decision Tree AUC: 0.9761
Higher AUC = better model at separating classes
```

LAB: PREDICT DIABETES- PIMA INDIANS DATASET

8 health measurements → predict whether a patient has diabetes (0 = healthy, 1 = diabetic). 768 rows, classic benchmark dataset.

Column Name	Description	Unit / Values
Pregnancies	Number of times the patient was pregnant	Count
Glucose	Plasma glucose concentration after 2 hours (OGTT test)	mg/dL
BloodPressure	Diastolic blood pressure	mm Hg
SkinThickness	Triceps skinfold thickness	mm
Insulin	2-hour serum insulin level	mu U/ml
BMI	Body Mass Index	kg/m ²
DiabetesPedigreeFunction	Genetic risk score for diabetes	0 – 2.5
Age	Age of the patient	Years
Outcome	Target column → 1 = Diabetic, 0 = Healthy	Binary (0/1)

Step 1 — Load & Explore

```
# ■ Load and Explore ■
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, roc_auc_score, roc_curve)

# Download the dataset if not already present
!test -f diabetes.csv || wget https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv

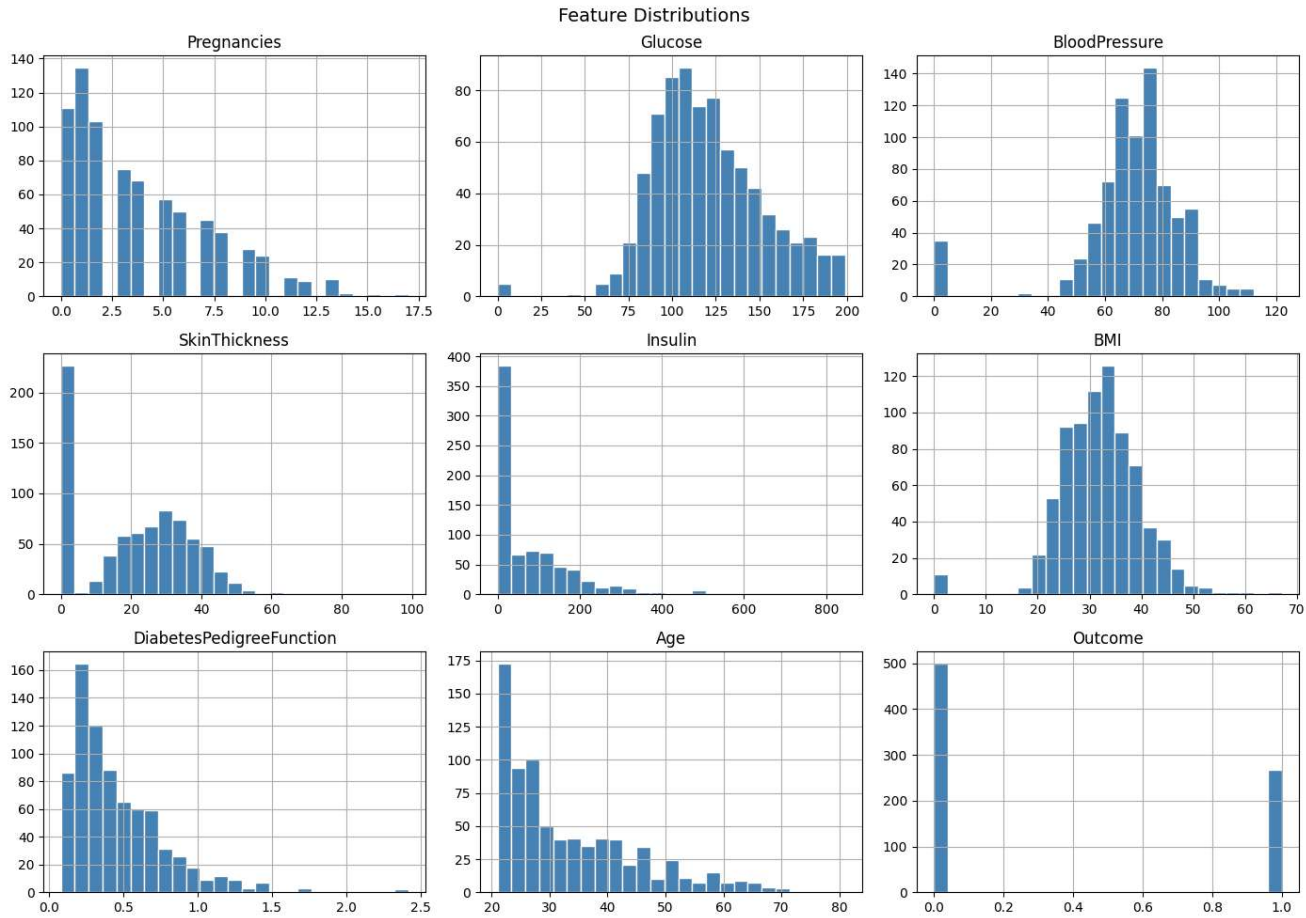
df = pd.read_csv('diabetes.csv') # https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database
print(df.shape) # (768, 9)
```

```
print(df['Outcome'].value_counts()) # 500 healthy, 268 diabetic
print(df.describe().round(2)) # stats: mean/std/min/max
# Distribution of each feature vs outcome
df.hist(bins=25, figsize=(14,10), color='steelblue', edgecolor='white')
plt.suptitle('Feature Distributions', fontsize=14); plt.tight_layout(); plt.show()
```

```
(768, 9)
Outcome
0    500
1    268
Name: count, dtype: int64
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
count	768.00	768.00	768.00	768.00	768.00	768.00
mean	3.85	120.89	69.11	20.54	79.80	31.99
std	3.37	31.97	19.36	15.95	115.24	7.88
min	0.00	0.00	0.00	0.00	0.00	0.00
25%	1.00	99.00	62.00	0.00	0.00	27.30
50%	3.00	117.00	72.00	23.00	30.50	32.00
75%	6.00	140.25	80.00	32.00	127.25	36.60
max	17.00	199.00	122.00	99.00	846.00	67.10

	DiabetesPedigreeFunction	Age	Outcome
count		768.00	768.00
mean	0.47	33.24	0.35
std	0.33	11.76	0.48
min	0.08	21.00	0.00
25%	0.24	24.00	0.00
50%	0.37	29.00	0.00
75%	0.63	41.00	1.00
max	2.42	81.00	1.00



Step 2 — Clean (Replace Impossible Zeros)

```
# ■■ Handle Missing Values ■■
# Glucose, BMI, etc. cannot be 0 - these are missing values disguised as zeros
fix_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in fix_cols:
    n = (df[col] == 0).sum()
    df[col] = df[col].replace(0, df[col].median())
    print(f'{col:25s}: {n} zeros → replaced with median {df[col].median():.1f}')
```

```
Glucose          : 5 zeros → replaced with median 117.0
BloodPressure    : 35 zeros → replaced with median 72.0
SkinThickness    : 227 zeros → replaced with median 23.0
Insulin          : 374 zeros → replaced with median 31.2
BMI              : 11 zeros → replaced with median 32.0
```

Step 3 — Split, Scale, Train

```
# ■■ Split, Scale, Train ■■
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y # stratify keeps class ratio
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # fit on train, transform train
X_test = scaler.transform(X_test) # only transform test (no leakage!)
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
# Print learned coefficient for each feature
for name, coef in zip(X.columns, model.coef_[0]):
    print(f'{name:30s}: {coef:+.3f}') # positive → increases diabetes risk
```

```
Pregnancies      : +0.376
Glucose           : +1.209
BloodPressure     : -0.054
SkinThickness     : +0.029
Insulin           : -0.133
BMI               : +0.701
DiabetesPedigreeFunction : +0.245
Age               : +0.138
```

Step 4 — Evaluate: Accuracy, Confusion Matrix, ROC

```
# ■■ Evaluate ■■
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
print(classification_report(y_test, y_pred, target_names=['Healthy', 'Diabetic']))
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm, display_labels=['Healthy', 'Diabetic']).plot(cmap='Blues')
plt.title('Diabetes Prediction - Confusion Matrix'); plt.show()
# ROC + AUC
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)
plt.figure(figsize=(6.5, 4.5))
plt.plot(fpr, tpr, lw=2.5, color='steelblue', label=f'AUC = {auc:.3f}')
plt.plot([0,1],[0,1], 'k--'); plt.xlabel('FPR'); plt.ylabel('TPR')
plt.title('Diabetes Model ROC Curve'); plt.legend(); plt.tight_layout(); plt.show()
# Expected: Accuracy ~78%, AUC ~0.84
```